EECS 388: Embedded Systems Lab

Final Lab Project

Objective:

In this project, you will have the opportunity to apply the knowledge and skills you have acquired in the previous labs. You will also have the opportunity to make some design choices in how you implement the final lab project.

There are two options for the Final Lab Project, like Lab08.

Option 1 – Simple Embedded System – Raspberry PI and HiFive

This project centers around designing a simple embedded system that encompasses automatic braking and automatic steering functionality for a self-driving car. You will utilize the Raspberry PI and HiFive boards along with embedded components (serial communication, sensors and actuators) that you have encountered so far to create a prototype that is capable of distinguishing between different braking scenarios (obstacles which are far away, near or very near) and is also able to simulate automatic steering based on a video input using a pre-trained DNN (Deep Neural Network) model.

Option 2 – Simple Embedded System – Two Arduinos

This project centers around designing a simple embedded system that encompasses automatic braking and automatic steering functionality for a self-driving car using all of the sensors that we've worked on up until this point. You will utilize the Arduino boards along with embedded components (serial communication, sensors, and actuators) that you have encountered so far to create a prototype that is capable of distinguishing between different breaking scenarios (obstacles which are far away, near, or very near) and is able to simulate steering using pre-determined angle values.

Project Summary:

1. **Team Composition:** This final project is designed for groups of **2 students**. Each team member must be from the **same** lab section.
2. **Total Marks:** This project will be evaluated out of **50 points**.
3. **Project evaluation:** Your project will be evaluated based on the following parts:
   a. **Part 1 (35 points):** Part 1 comprises four tasks. Each task carries 8.75 points.

      i. **Task 1:** TFmini LiDAR
      This task involves using the TFmini Lidar sensor to collect distance measurements for objects within your surroundings. Drawing from your knowledge gained in Labs 04, you will modify the starter code to collect distance values.
      ii. **Task 2:** LED Braking
      For this task, you will use the distances values to update LEDs to indicate a certain distance. You will use knowledge gained from lab02 for this task.

iii. **Task 3:** Board to Board Communication
    1. Option 1 (Using RaspberryPi/HiFive) - For this task, you will execute the **dnn.py** script from Lab 07 to utilize a pre-trained DNN model. This model uses video frames (available in the provided directory) to infer the steering angle. Your challenge is to adapt the **dnn.py** script to **transmit this steering angle** to the HiFive board using the board-to-board UART connection knowledge you have acquired in Lab 09-option1.
    2. Option 2 (Using Arduinos) – For this task, will take the distance values obtained from the TFmini LiDAR from Task1 and **transmit the distance values** to the other Arduino board.

    This task will use knowledge you gained from lab09.

iv. **Task 4:** Steering with Servo Motor
    In this task, you must use the received angle values to drive a servo motor. Update the file from Task 1 and write the code for driving the servo using the angle values received from Raspberry Pi. Use your understanding of Lab 05 to implement this part.

b. **Part 2 (15 points):** This part requires you to prepare a short presentation, up to 10 minutes maximum, where you talk about your project. You may use Microsoft PowerPoint, presentation board, or another tool. You do not have to demo during your presentation if you've already completed your demo with your GTA. Your presentation should cover:
    i. Your understanding of the tasks
    ii. An explanation of how your code works
    iii. Any issues encountered while working on the project and how you overcame them.
    iv. Any improvements to the code you would make in the future and how to start on those changes.

<u>Project Deliverables:</u>

1. **Code:** all members of the team should submit the same **code** files to Canvas. Without the code submission, we will be unable to grade you. Submit the files directly, do not zip them.
2. **Demo:** A demonstration of all the components working together and producing the expected output. A video is sufficient for this, but be sure that you have both team members in the video!
3. **Presentation:** A short presentation, up to 10 minutes maximum.

<u>Deadlines:</u>

The deadlines for the two parts are given below:

- **Part 1:** The demo for Part 1 is due 2 weeks from the start of final project (the start for each team is their respective lab sessions). The demo can be given any time within this period if the team has completed their project. You can also show your recorded demo videos during your presentation if you haven't yet shown the GTA.

- **Part 2:** The presentation for Part 2 is due during lab in week 3. The presentation cannot be given earlier than this. The day of the presentation is also the last day for the demo.

We recommend that teams get their demos done ahead of time so that there is no rush on presentation day. **You can record a video for your demonstration if it shows all team members**. There will be no extra time given to setup and demo during the presentation.

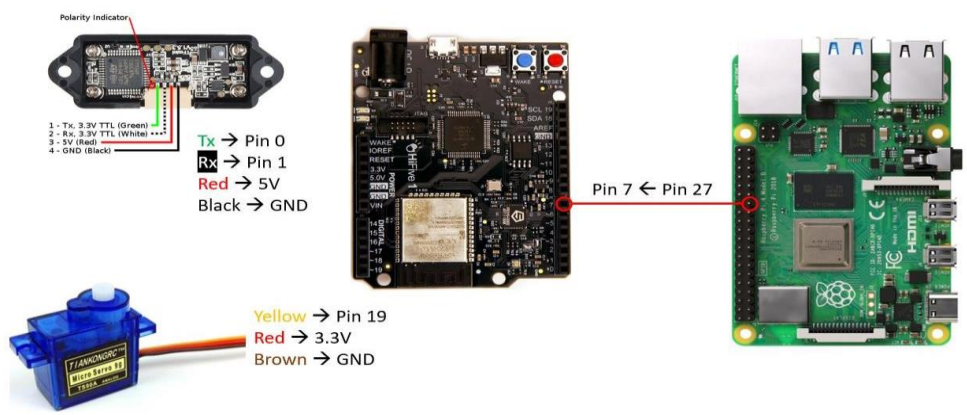
Here is a rough timeline that is recommended to stay on track:

- Week 1 (April 6-10) – Complete task 1 & 2 during your lab session. Review tasks 3 & 4.
- Week 2 (April 13-17) – Complete task 2 & 3 during your lab session. Demo to your GTA.
- Week 3 (April 20-24) – Final Demo (if necessary) and Code is due for tasks 1-4. Present presentation during your lab session.

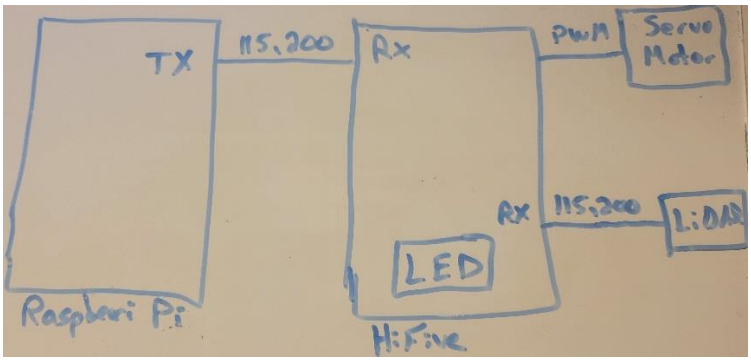
<u>Necessary Equipment (Option 1):</u>

1. HiFive Board
2. TFmini Lidar Sensor
3. Servo Motor
4. Raspberry Pi

<u>Connection Diagram (Option 1):</u>



<u>Block Diagram (Option 1)</u>:



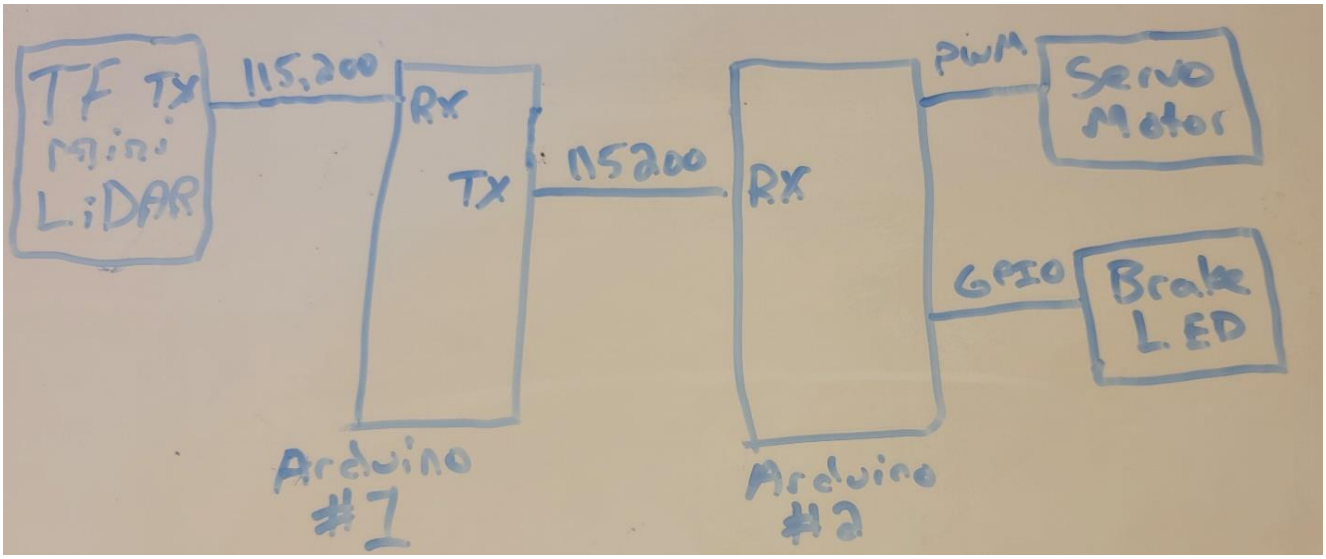
Necessary Equipment (Option 2):

1. Arduino Board x 2
2. TFmini Lidar Sensor
3. Servo Motor
4. Breadboard
5. LEDs, resistors

Connection Diagram (Option 2):

1. TFmini Lidar
    - a Green TX to Arduino1 RX
    - b Red 5V to Arduino1 5V
    - c Black GND to Arduino1 GND
2. ServoMotor
    - a Yellow PWM-> Arduino2 GPIO_6
    - b Red Power -> Arduino2 5V
    - c Brown GND -> Arduino2 GND
3. Breadboard
    - a Utilize LED circuit from Lab02
4. Arduino_1 TX to Arduino_2 RX
5. LEDs, resistors

Block Diagram (Option 2):



Task Details:

There are two compressed source files for the final project. The **final_project_option1.zip** contains the source code meant to run in the **HiFive** folder and the **DNN** folder has the source code which needs to run on the Raspberry Pi. The **final_project_option2.zip** contains the source code meant to run on the Arduinos. Be sure to use the correct starter code based on the option you select.

Task 1: Distance with LiDAR – auto_brake()

Task 1 asks you to implement a prototype automatic braking system using the TFmini lidar sensor from Lab 04. The idea behind this task is to measure the distance between our sensor (and therefore our hypothetical car) and any obstacles ahead in the direction of the sensor and decide if brakes need to be applied and how hard. You will need to configure the UART for 115200 baud speed to read distance values from the LiDAR.

Option 1: Raspberry PI/HiFive

You will still be able to re-use most of your lab04 LiDAR code for this task. Only minimal modifications are necessary.

Option 2: Arduino

This will need to be done in the arduino1_lidar starter code.

Task 2: LED Breaking Indicator – auto_brake()

For the purposes of this project, we define any object that is further than 200 cm as a safe distance away and no braking is needed. If any object is less than 200 cm but more than a 100 cm away, we assume that it is getting close enough to warrant braking, but only **lightly**. For anything less than 100 cm away, we assume that the object is getting too close to the vehicle, and the brakes need to be applied in full force. Finally, for something that is less than 60 cm away, we assume it is too close to us to keep the vehicle in motion and so it needs to stop and be stationary. To simulate braking, you will change the color of the LED to indicate the level of braking. The levels of braking, and their representation using LED colors, are detailed below:

Distance	Action	First LED Indicator
> 200 cm	Safe distance, no braking	Turn ON Green LED
100 cm < d ≤ 200 cm	Close, brake lightly	Turn ON Yellow LED (Red+Green)
60 < d ≤ 100 cm	Very close, break hard	Turn ON Red LED
d ≤ 60	Too close, must stop	Turn ON *flashing* Red LED (Flashing period = 100ms)

Option 1: Raspberry PI/HiFive

You will be able to directly test this on the HiFive board with the received distance from the LiDAR just like you did in Lab04.

Option 2: Arduino

This task will need done on the arduino2_steering_brake starter code. You will need to use test values for distance to check this until you complete Task 3 (receiving the distance from arduino1_lidar).

Task 3: Board to Board Communication

Option 1: Raspberry PI/HiFive – Send the steering angle from the Pi

To simulate our car's automatic steering, we will use the DNN model from Lab 09. If you recall, the DNN model predicted the turning angles in degrees based on the input video footage. The script **dnn.py** runs the inferencing using the pretrained model with the video frames as input and produces a degree value as prediction and stores this in a variable called **deg** in the python file.

It will be your task to modify **dnn.py** so that the python script sends this predicted angle value to the HiFive from the Pi using the UART connection, instead of simply printing it to the terminal.

To initiate a serial (i.e., UART) connection from a python script, you will need the **serial** library. That has already been added to the **dnn.py** file. You would first need to instantiate a new serial connection and store it in a variable. A new connection can be created using the following command, and stored in a variable called **ser**, like so:

ser = serial.Serial("<path to connection file>", baudrate)

The path to the connection file is the ttyAMA*x* (where *x* is 1 or 2 or 3) file that you used in Lab 09. We recommend you use ttyAMA1, although the project can be accomplished using ttyAMA2 as well. The *baudrate* is the same as what you had to use in the Lab 09 **screen** command.

Once established, you need to actually write to that connection, so that it can be sent by the Pi and received by the HiFive. To do that, you can use the **write()** method of a serial object. For example:

ser.write()

Keep in mind, however, that passing any python objects directly might not give you the correct output on the other end. It is a good idea to first convert your input to **write()** into a byte format. You can do that using the **bytes()** function in Python. You can run the **dnn.py** file the same way you did in Lab 07.

For task 3 you will need to connect the HiFive with the Pi via UART so that they can talk to each other. Use the information given in Lab 09 to wire up the HiFive and Pi; remember you only need the HiFives UART1 to connect with the Pi. UART0 will be used for the lidar.

Once connected, you need to modify your **eecs388_final.c** file to receive any incoming bytes from the Pi and extract the angle value. Once again, your Lab 09 code here will help, but you may need to do some extra steps. You can, for e.g., use the **sscanf()** function to extract the data, however this is only a suggestion and there are many other ways to go about it. Here is more info on **sscanf**:

- https://en.cppreference.com/w/c/io/fscanf
- https://www.tutorialspoint.com/c_standard_library/c_function_sscanf.htm

Option 2: Arduinos – Send distance from Arduino1 to Arduino2

You will need to write code on the Arduino1 to transmit the distance value to Arduino2. You have several design decisions to make here:

- How frequently to send the data. You can choose to send every value received from the LiDAR, or you can setup a timed task to only send it at a periodic rate (for example, 5Hz/5 times per second/every 200ms).
- How to format and send the data.
    - You can send it the same way the LiDAR does (9 bytes per frame)
    - You can choose to send only the distance_low, distance_high byte
    - You can choose to send just a single ASCII character that represents the distance, example: 'u' (Ascii value of 117 decimal to represent 117cm)

You will also need to write code on the Arduino2 to receive the distance value. You can do this with polling techniques, but remember that the Arduino2 has other things that it will need to accomplish in its main loop.

Task 4:

This task will involve getting the servomotor setup to turn to a specific angle value. To test the function and confirm the servomotor is working, try to just turn it between 0 and 180 degrees. You can leverage your code from Lab05 for this, but remember that the main loop has other things it needs to accomplish, so using delay() functions may affect other timings.

With all this done, you should have a simple prototype for automatic braking and steering!

For the demo, you will be asked to show that your LEDs show the right color based on distance from some object, and that your servo motor responds accordingly to the specified angle values.

Option 1: Raspberry PI/HiFive

After successfully extracting the angle value from the PI, you must then pass it to the **steering()** function, which will turn the servo motor. To implement the steering function, you may look at your Lab 05 code.

Option 2: Arduinos –

You will need to setup an array of angle values in your Arduino code. You will loop through these values and use them as angles to drive the servomotor. Setup the following angles to loop through. Angle_values = [10, 25, 75, 45, 100, 40, 125, 15, 150, 50, 170]. These values will not be setup for you in the startup code, you will have to implement this so that it rotates through all the steering angles in this list and repeats back at the beginning.

Bonus Tasks

There are several different bonus tasks available for lab10. You can choose to mix and match these however you want, but it is **highly recommended** that you get the basic pieces of your final project working first. Most of these bonus tasks won't have much starter code, you will have to go research and implement it yourself!

Bonus 1 (For either HiFive or Arduino) (**2 points**) – Implement the flashing RED LED using a timer interrupt from Lab 9. This option is slightly easier to implement if you choose the HiFive option because you can use your Lab8 as a reference for implementing a 100ms timer. Instead of changing the LEDs during the auto_brake() function, you will just set their state value (Green, Yellow, Red, or flashing Red). When the 100ms timer goes off, you will update the LEDs based on the LED state.

Bonus 2 (For either the HiFive or Arduino) (**2 points**) – The technique we used to control the servomotor in lab05 was manually toggling the GPIO pin to generate a PWM signal. Implement a timer-based PWM generated signal that avoids using delay() function APIs.

Bonus 3 (For either the HiFive or Arduino) (**5 points**) – Implement a RS232 serial interrupt service routing to handle the LiDAR communication. This will require you to research external interrupt handling for the microcontroller you are using.

Bonus 4 (For the HiFive or Arduino) (**5 points**) – Implement the RS232 UART communication between the two microcontrollers by using an interrupt service routine on the HiFive board. This will require you to research external interrupt handling on the HiFive board (but you have a start on this from the lab08 code).

Bonus 5 (For electrical engineers!) (**3 points**) – Implement a dedicated 5V regulated power rail for our breadboards to support the peripherals. Determine about how much current our microcontroller can supply and how much is used by the TFmini LiDAR and servomotor. Demonstrate the peripherals are powered from the 5V rail and not from the microcontroller board.

Bonus 6 (For either the HiFive or Arduino) (**3 points**) – Implement an engine temperate sensor using the BMP180 temperature sensor from Lab06. You will configure the I2C connection to read the temperature sensor value, and configure a secondary LED circuit to indicate the state of the engine based on the table below:

Temperature	**Action**	**Second LED Indicator**
< 70 degrees Fahrenheit	Cold Engine – Drive gently	Turn ON Blue LED
70 degrees < temp ≤ 80	Normal condition	Turn ON Green LED
Temp > 80	Engine Overheat	Turn ON Red LED

Submission – 50 points

Attendance: Attendance is required for Week 1 and Week 3 (presentation week). If you complete the tasks during Week 1 or outside of normal lab hours, you do not need to attend the lab during week2.

Demo and Code: To obtain 35 points for this lab, you need to complete the code for tasks 1-4. Show the complete project to your GTA when you have each piece working individually and when you have it working altogether.

Regarding video demos:

- You can use a video of each piece working individually for partial credit, but to receive full credit everything has to be working together at the same time.
- Be sure your GTA knows it's you! Be sure you and your partner are visible in the video. If a GTA can't confirm its you, then you won't get credit for the demo video.

Option1:
Submit your modified eecs388_final.c and dnn.py code on the "Assignments" section on Canvas under the Lab 10 submission link. Make sure you have demonstrated your lab to your GTA before you submit your code. Screenshots/PDF/Text files will not be accepted. You need to submit the modified eecs388_final.c file and dnn.py file only.

Option2:
Submit your modified eecs_388_final1.cpp and eecs_388_final2.cpp code on the "Assignments" section on Canvas under the Lab 10 submission link. Make sure you have demonstrated your lab to your GTA before you submit your code. Screenshots/PDF/Text files will not be accepted. You need to submit the modified files only.

Presentation: To obtain 15 points for this lab, you will need to prepare a short presentation and present during your final lab session as specified above

Bonus:
Early completion Bonus: There is no early completion bonus for this lab

Other bonus tasks are detailed above.